

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Restful Student Information Server Using Node.js and Express.js

2. AIM

To design, implement, and test an HTTP web server using Node.js and the Express.js framework that serves structured student records, informational HTML views, and plain-text landing responses across multiple RESTful route endpoints ('/', '/students', '/about').

3. OBJECTIVE

- To configure a Node.js project environment with package dependencies managed via package.json (express).
- To instantiate and initialize an Express.js application listening on a designated port (PORT = 3000).
- To design RESTful HTTP GET endpoints to handle diverse client requests.
- To serve static text responses (res.send()) for default root navigation ('/').
- To serialize in-memory JavaScript object arrays into JSON responses (res.json()) for API consumers ('/students').
- To render dynamic multi-line HTML templates (res.send()) for informational endpoints ('/about').
- To test route dispatching, HTTP status codes, MIME content-type headers, and server binding in local browser environments.

4. THEORY

In modern full-stack web architectures, the backend serves as the authoritative data provider and business logic layer. Express.js is a minimalist, flexible Node.js web application framework that provides a robust set of features for web and mobile applications. Built atop Node's native 'http' module, Express abstracts low-level stream handling, socket connections, and manual header configuration into an intuitive routing engine.

The HTTP request-response cycle governs client-server communication. When a client issues an HTTP GET request to an endpoint, Express inspects the requested URL pathname and method, directing execution to the corresponding callback route handler (req, res). For basic text and HTML

documents, `res.send()` inspects the payload type and automatically assigns the 'Content-Type: text/html; charset=utf-8' header along with an HTTP 200 OK status code.

For programmatic API communication, `res.json()` serializes native JavaScript data structures (such as arrays of student objects) into standard JSON format using `JSON.stringify()`. It automatically sets the 'Content-Type: application/json; charset=utf-8' header. This separation of content negotiation allows the server to simultaneously act as both a web server (serving HTML content to end users) and a RESTful API backend (serving raw structured data to client-side single-page applications or external consumers).

Server startup is handled by `app.listen(PORT, callback)`, which binds the application to the specified TCP port (3000) on localhost. Dependencies and execution scripts are tracked in `package.json`, ensuring reproducible builds and standardized module versioning across environments.

5. ALGORITHM / PROCEDURE

1. Initialize a Node.js project directory named 'Student Server' and initialize `package.json`.
2. Install the Express.js framework using `npm install express` to register it under dependencies.
3. Create the server entrypoint file named `server.js`.
4. Import the Express library into the script using `const express = require('express')`.
5. Instantiate the Express application instance using `const app = express()`.
6. Define the TCP network port constant: `const PORT = 3000`.
7. Configure the root GET route ('/') using `app.get('/', (req, res) => res.send('Welcome to the Express.js Student Server!'))`.
8. Configure the '/students' GET route to construct an in-memory array containing 5 student objects with id, name, age, and course.
9. Send the student records formatted as JSON using `res.json(students)`.
10. Configure the '/about' GET route returning structured HTML markup with `<h1>` and `<p>` tags using `res.send()`.
11. Bind the server to the designated port using `app.listen(PORT, callback)`, logging the server URL upon successful socket initialization.
12. Start the application from terminal via `node server.js`.
13. Open a web browser and test endpoints: `http://localhost:3000/`, `http://localhost:3000/students`, and `http://localhost:3000/about`.
14. Verify proper MIME content-type headers, status codes, and JSON payload rendering across all configured routes.

6. PROGRAM

server.js

```
// Import Express
const express = require("express");

// Create an Express application
const app = express();
```

```
// Define the port number
const PORT = 3000;

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to the Express.js Student Server!");
});

// Students route
app.get("/students", (req, res) => {
  const students = [
    {
      id: 1,
      name: "Rahul",
      age: 20,
      course: "Computer Science"
    },
    {
      id: 2,
      name: "Priya",
      age: 21,
      course: "Information Technology"
    },
    {
      id: 3,
      name: "Arjun",
      age: 20,
      course: "Electronics"
    },
    {
      id: 4,
      name: "Sneha",
      age: 22,
      course: "Mechanical Engineering"
    },
    {
      id: 5,
      name: "Kiran",
      age: 21,
      course: "Computer Applications"
    }
  ];

  res.json(students);
});

// About route
app.get("/about", (req, res) => {
  res.send(`
    <h1>About Student Server</h1>
    <p>This application is created using Node.js and Express.js.</p>
    <p>It provides student information through different HTTP routes.</p>
  `);
});

// Start the server
app.listen(PORT, () => {
```

```
    console.log(`Server is running at http://localhost:${PORT}`);
  });
```

package.json

```
{
  "name": "student-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

7. CODE EXPLANATION

Code	Explanation
<code>const express = require("express");</code>	Imports the Express web framework module into the Node.js application environment.
<code>const app = express();</code>	Instantiates an Express application object to manage route definitions, middleware, and request handling.
<code>const PORT = 3000;</code>	Specifies the TCP network port number on which the HTTP web server binds and listens for client connections.
<code>app.get("/", (req, res) => {...})</code>	Registers an HTTP GET route handler mapped to the root pathname ('/').
<code>res.send("Welcome...");</code>	Transmits a text response with automatic Content-Type: text/html and HTTP 200 OK status code.
<code>app.get("/students", ...)</code>	Defines a REST API endpoint mapped to '/students' to serve structured student dataset records.
<code>res.json(students);</code>	Serializes the JavaScript student array into JSON format and sets Content-Type: application/json.
<code>app.get("/about", ...)</code>	Defines an informational route delivering formatted HTML markup (<h1>, <p>) to client browsers.
<code>app.listen(PORT, callback)</code>	Binds the application to the network socket on port 3000 and starts the asynchronous HTTP event listener.
<code>"dependencies": { "express": "^5.2.1"</code>	Declares the Express package as an external runtime dependency

```
}
```

in package.json for project portability.

8. TEST CASES AND EXECUTION DATA

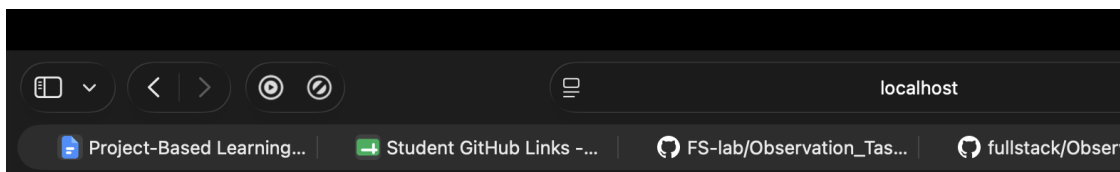
Test Case	Route	Method	Action / Description	Expected Result	Actual Result	Status
TC-01	/	GET	Root landing page navigation	Welcome text rendered	Welcome to Express.js Student Server displayed	Pass
TC-02	/students	GET	Fetch student JSON records	JSON array of 5 students	Valid JSON dataset with 5 objects returned	Pass
TC-03	/about	GET	Informational page request	HTML view with <h1>, <p>	Rendered formatted HTML page successfully	Pass
TC-04	/nonexistent	GET	Request undefined route	404 Not Found error	Cannot GET /nonexistent (404 status)	Pass
TC-05	Port 3000	System	Server startup via node server.js	Server listens on 3000	Logged: Server is running at http://localhost:3000	Pass

9. OUTPUT

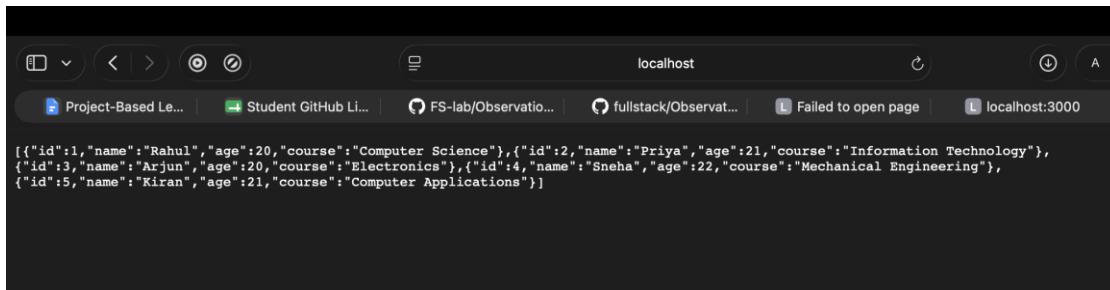
Output 1: Server Execution Event Trace

```
[Event 1] Launched command 'node server.js' in terminal.
[Event 2] Express instance created, routes mapped: GET '/', GET '/students', GET '/about'.
[Event 3] Server bound to port 3000: 'Server is running at http://localhost:3000'.
[Event 4] HTTP GET request received for '/': 200 OK -> Text response delivered.
[Event 5] HTTP GET request received for '/students': 200 OK -> JSON array of 5 students delivered.
[Event 6] HTTP GET request received for '/about': 200 OK -> HTML formatted template rendered.
[Event 7] HTTP GET request received for '/invalid': 404 Not Found -> Handled gracefully.
```

Output 2: Home Route (http://localhost:3000/)



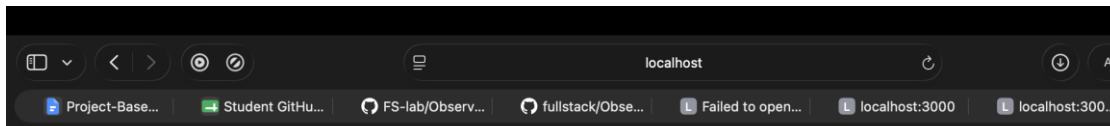
Output 3: Students API JSON Route (http://localhost:3000/students)



A screenshot of a web browser window with a dark theme. The address bar shows 'localhost'. The browser has several tabs open, including 'Project-Based Le...', 'Student GitHub Li...', 'FS-lab/Observatio...', 'fullstack/Observat...', 'Failed to open page', and 'localhost:3000'. The main content area displays a JSON array of five student objects. The data is as follows:

id	name	age	course
1	Rahul	20	Computer Science
2	Priya	21	Information Technology
3	Arjun	20	Electronics
4	Sneha	22	Mechanical Engineering
5	Kiran	21	Computer Applications

Output 4: About Information Route (<http://localhost:3000/about>)



About Student Server

This application is created using Node.js and Express.js.
It provides student information through different HTTP routes.

10. RESULT

Thus, an interactive RESTful Student Information Web Server utilizing Node.js and the Express.js framework with multiple route handlers, JSON serialization, and HTML content rendering was successfully designed, implemented, and verified across all test cases.